- Optimistic Locking
- Pessimistic Locking

## 11.1 Optimistic Locking

Optimistic locking allows multiple users to access the same record for edits, and assumes a minimum of conflicts with the data. It does this by checking whether another process has made changes to a record since it was opened. An `ActiveRecord::StaleObjectError` exception is thrown if that has occurred and the update is ignored.

**Optimistic locking column**

In order to use optimistic locking, the table needs to have a column called `lock_version` of type integer. Each time the record is updated, Active Record increments the `lock_version` column. If an update request is made with a lower value in the `lock_version` field than is currently in the `lock_version` column in the database, the update request will fail with an `ActiveRecord::StaleObjectError`. Example:

```
c1 = Client.find(1)
c2 = Client.find(1)

c1.first_name = "Michael"
c1.save

c2.name = "should fail"
c2.save # Raises an ActiveRecord::StaleObjectError
```

You're then responsible for dealing with the conflict by rescuing the exception and either rolling back, merging, or otherwise apply the business logic needed to resolve the conflict.

This behavior can be turned off by setting `ActiveRecord::Base.lock_optimistically = false`. To override the name of the `lock_version` column, `ActiveRecord::Base` provides a class attribute called `locking_column`:

```
class Client < ActiveRecord::Base
  self.locking_column = :lock_client_column
end
```

## 11.2 Pessimistic Locking

Pessimistic locking uses a locking mechanism provided by the underlying database. Using `lock`when building a relation obtains an exclusive lock on the selected rows. Relations using `lock` are usually wrapped inside a transaction for preventing deadlock conditions.

For example:

```
Item.transaction do
  i = Item.lock.first
  i.name = 'Jones'
  i.save
end
```

The above session produces the following SQL for a MySQL backend:

```
SQL (0.2ms)   BEGIN
Item Load (0.3ms)   SELECT * FROM `items` LIMIT 1 FOR UPDATE
Item Update (0.4ms)   UPDATE `items` SET `updated_at` = '2009-02-07
18:05:56', `name` = 'Jones' WHERE `id` = 1
SQL (0.8ms)   COMMIT
```